

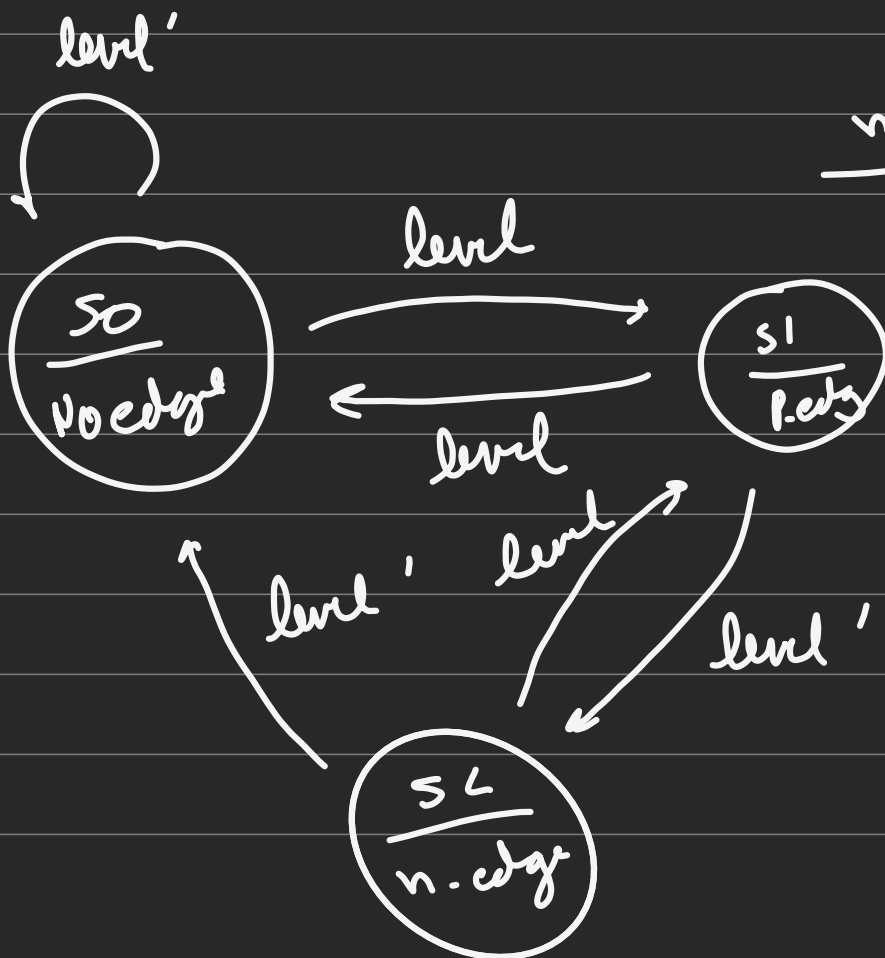
Edge detection FSM

<u>Input</u>	<u>output</u>	
level	P. edge	0 $\rightarrow$ 1
	N. edge	1 $\rightarrow$ 0
	- edge	

mealy



moore



OR

```
reg level_prev;  
always @ ( _____ )  
    if (~reset_n)  
        level_prev <= 1'b0;  
    else  
        level_prev <= level;
```

assign tve edge = (~level_prev) & level;
- -ve edge = (level_prev) & (~level);
- edge = level_prev ^ level;

Debounce FSM

noisy signal

timer, while signal is bouncing

input noisy, timer done

output timer.reset, debounced

when no need of timer when clean, stable output

s_0/s_2 s_2/s_3

reg [3:0] _____
parameter s0 = _____

// state register logic

// next state logic

s0: if noisy is low (button not pressed):
 stays at s0;

if noisy goes high (button pressed):
 moves to s1 (debounced press).

s1: if noisy goes low (button bounce):
 return to s0;

if noisy stays high but timer not done:
stay in S1

if noisy stays high & timer done:
moves to S2 (stable pressed)

S2: if noisy goes low (button released):
moves to S3 (debounce release)

if noisy stays high:
remain in S2;

S3: if noisy goes high (button bounce):
returns to S2:

if noisy stays low but timer not done:
stay in S3;

if noisy stays low & timer done:
return to S0

